

Incident-Playbook

Woche 1 - Kritische Blocker

1. Schnellreferenz

Incident	Schwere	Erste Massnahme	Escalation
Postgres nicht erreichbar	CRITICAL	Connection String prüfen	DBA / Neon Support
Redis Rate Limit fail	HIGH	Circuit Breaker aktivieren	DevOps
Stripe Webhook Fehler	HIGH	Signature validieren	Payment Team
Audit Log Integritat	MEDIUM	Hash-Chain verifizieren	Security
Backup fehlgeschlagen	HIGH	Manuelles Backup	DevOps

2. Notfall-Kontakte

Rolle	Name	Telefon	Verfügbarkeit
Incident Commander	[Name]	[Nummer]	24/7
DBA / Database	[Name]	[Nummer]	Business Hours
DevOps / Infrastructure	[Name]	[Nummer]	Business Hours
Security	[Name]	[Nummer]	Business Hours
Neon Support	support@neon.tech	Slack	24/7
Upstash Support	support@upstash.com	-	Business Hours
Stripe Support	dashboard.stripe.com/support		24/7

3. Incident Playbooks

3.1 PostgreSQL Ausfall

Symptome

- Verbindungsfehler: 'connection refused' oder 'timeout'
- Langsame Queries (>5s)
- Hohe CPU/RAM-Auslastung auf DB-Instanz
- Fehlgeschlagene Transaktionen

Diagnose-Schritte

Schritt	Befehl/Aktion	Erwartetes Ergebnis
1. Connection prüfen	psql \$DATABASE_URL -c "SELECT 1"	Erfolgreich
2. Connection Pool Status	Prisma Studio > Connections	< Pool Size
3. Long Running Queries	SELECT * FROM pg_stat_activity WHERE state = 'active';	> 30s
4. Disk Space	SELECT pg_size_pretty(pg_database_size('cargobike'));	> 80%
5. Neon Console	console.neon.tech > Metrics	Normal

Losungsstrategien

Problem	Losung	Zeit
Connection Pool erschöpft	App neu starten, pool_size erhöhen	5 min
Slow Query	EXPLAIN ANALYZE, Index erstellen	15 min
Disk Full	Neon: Auto-Scale aktivieren	10 min
Region-Ausfall	Failover zu Backup-Region	30 min
Complete DB Loss	Restore von S3 Backup	60 min

3.2 Redis Rate Limiting Ausfall

Symptome

- Alle Requests werden blockiert (429)
- Keine Requests werden blockiert
- Hohe Latenz bei Rate Limit Checks
- Redis Connection Errors in Logs

Diagnose-Schritte

Schritt	Befehl/Aktion	Erwartetes Ergebnis
1. Redis Erreichbarkeit	redis-cli -u \$REDIS_URL PING	PONG
2. Memory Usage	redis-cli INFO memory	used_memory < maxmemory
3. Connection Count	redis-cli INFO clients	connected_clients < limit
4. Key Count	redis-cli DBSIZE	Erwarteter Wert

5. Upstash Console	console.upstash.com	Status OK
--------------------	---------------------	-----------

Circuit Breaker Fallback

Bei Redis-Ausfall sollte die App automatisch in den Fallback-Modus wechseln. Konfiguration in .env:
RATE_LIMIT_FALLBACK=allow_all (nur für kurze Zeit!) oder RATE_LIMIT_FALLBACK=deny_all
(sicherer, aber User-Impact).

3.3 Stripe Webhook Fehler

Symptome

- Webhooks schlagen fehl mit 400 Bad Request
- Payments nicht in DB aktualisiert
- Stripe Dashboard zeigt fehlgeschlagene Webhooks
- Replay-Angriffe vermutet

Diagnose-Schritte

Schritt	Aktion	Erwartetes Ergebnis
1. Stripe Dashboard	Developers > Webhooks	Endpoint aktiv
2. Recent Deliveries	Click Endpoint > Recent	200 Status
3. Secret prüfen	Env: STRIPE_WEBHOOK_SECRET	whsec_xxx
4. Raw Body	Code Review: express.raw()	Vor JSON.parse
5. Idempotency	processedEvents Tabelle	Keine Duplikate

Replay-Angriff Erkennung

Wenn dieselbe Event-ID mehrfach verarbeitet wird: 1. Prüfe processedEvents Tabelle auf Duplikate. 2. Bei >1 Eintrag mit gleicher ID: Möglicher Replay-Angriff. 3. Webhook-Secret sofort rotieren! 4. Stripe Support kontaktieren.

3.4 Audit Log Integritätsverletzung

Symptome

- Hash-Chain Verifikation schlägt fehl
- Fehlende Einträge in audit_log Tabelle
- Verdacht auf Manipulation

Verifikations-Befehl

SELECT verify_audit_integrity() -- Gibt TRUE zurück wenn OK, FALSE wenn verletzt

Massnahmen bei Integritätsverletzung

1. Sofort Security-Team informieren
2. Alle Zugriffe auf audit_log Tabelle loggen
3. S3 Archive mit Datenbank abgleichen
4. Forensische Analyse der Abweichungen
5. Bei Bestätigung: Incident Report erstellen

3.5 Backup Fehlgeschlagen

Symptome

- Cron-Job fehlgeschlagen (Email Alert)
- Keine neuen Backups in S3
- S3 Bucket fast leer oder veraltet

Manuelles Backup

Sofort manuell ausführen:

```
pg_dump -Fc $DATABASE_URL > /tmp/manual_backup_$(date +%F).dump
```

```
aws s3 cp /tmp/manual_backup_*.dump s3://cargobit-backups/manual/
```

Restore testen (auf separater DB!):

```
pg_restore -d $TEST_DB_URL /tmp/manual_backup_*.dump
```

4. Escalation-Matrix

Schwere	Beispiel	Response Time	Kontaktieren
P1 - CRITICAL	DB komplett ausgefallen	<5 min	Incident Commander + DBA
P2 - HIGH	Redis nicht erreichbar	<15 min	DevOps on-call
P3 - MEDIUM	Backup fehlgeschlagen	<1 Stunde	DevOps
P4 - LOW	Performance Degradation	<4 Stunden	Dev Team

5. Post-Incident Prozess

1. Incident in Jira/Linear dokumentieren
2. Root Cause Analysis (RCA) innerhalb 48h
3. Action Items erstellen und priorisieren
4. Runbook aktualisieren falls notwendig
5. Postmortem-Meeting planen (bei P1/P2)
6. Lessons Learned im Team teilen

Postmortem Template

- **Datum**: [Incident Datum]
- **Dauer**: [Start] - [Ende] ([Gesamtzeit])
- **Schwere**: [P1/P2/P3/P4]

- **Zusammenfassung**: [Was ist passiert?]
- **Auswirkung**: [Welche User/Systeme betroffen?]
- **Ursache**: [Warum ist es passiert?]
- **Losung**: [Was wurde getan?]
- **Action Items**: [Was wird getan um Wiederholung zu verhindern?]